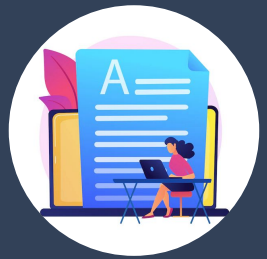
Basics of Text Processing 02

INTERVIEW
KICKSTART

Natural Language Processing - Roadmap

NLP 1 → **NLP 2** → **NLP 3**



- Word Embeddings
- Seq2Seq Modelling

- Transformers

- BERT

What will you learn today?

01 How is text represented? What are word embeddings?

02 Why do we need Word Embeddings?

03 What are the different types of Word Embeddings?

04 What are the different ways in which these word embeddings can be trained?

Setting Clear Expectations

How to get the most out of this session?

- Depth of Coverage
- Interaction
- Take notes
- Pre-requisites

Text Representation

Text Representation

- Bag of Words
- Term Frequency Inverse Document Frequency (TF-IDF)

Problems

- High dimensionality
- Sparse features
- Synonym words are treated differently: “awesome” and “amazing”

Text Embeddings

Word Embeddings

What?

Why?

How?

Word Embeddings

What?

Why?

How?

Word Embeddings: What?

(enhancing text representations)

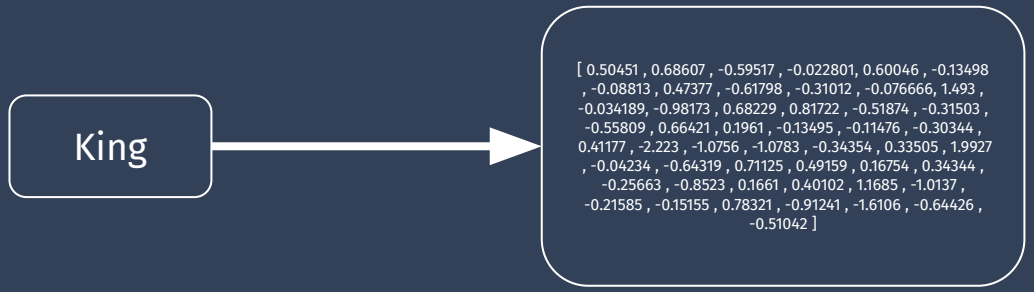
- Word embeddings capture semantic relationships more efficiently.

Instead of

Word Embeddings

(enhancing text representations)

- Whenever we say “word” or token in this context think about



Word Embeddings

What?

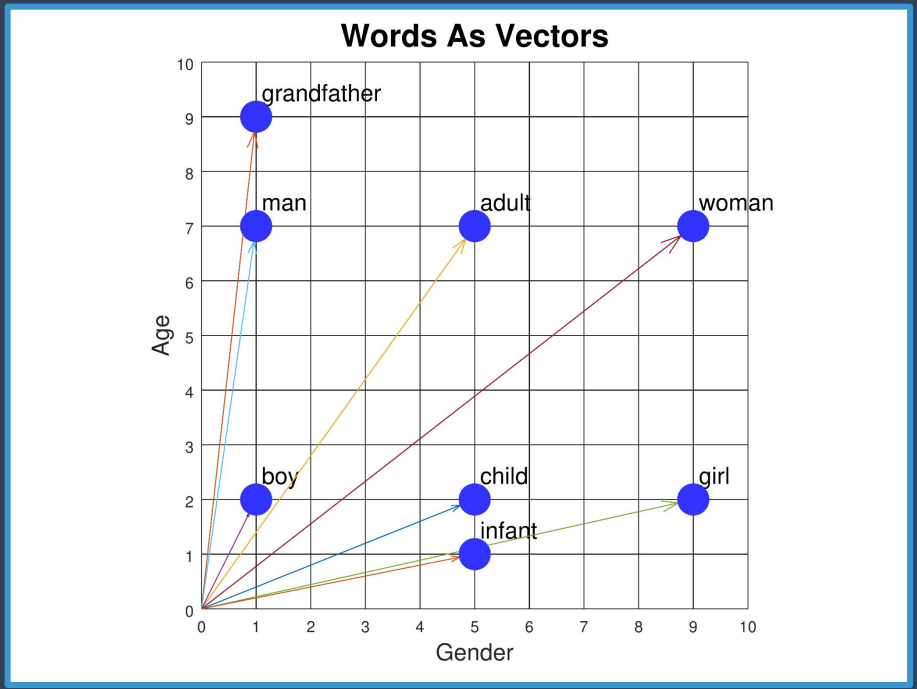
Why?

How?

Word Embeddings: Why?

(enhancing text representations)-

- Word embeddings capture semantic relationships more efficiently.

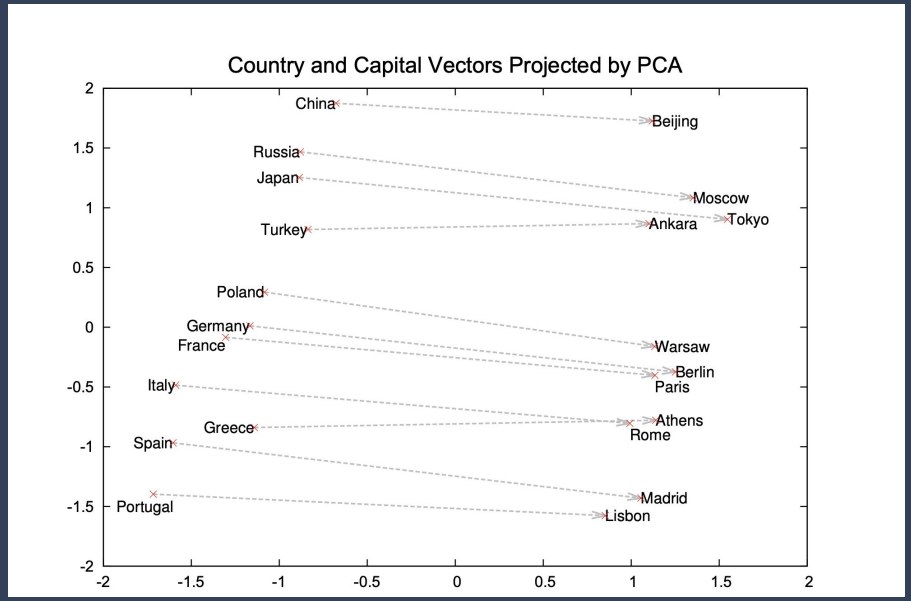


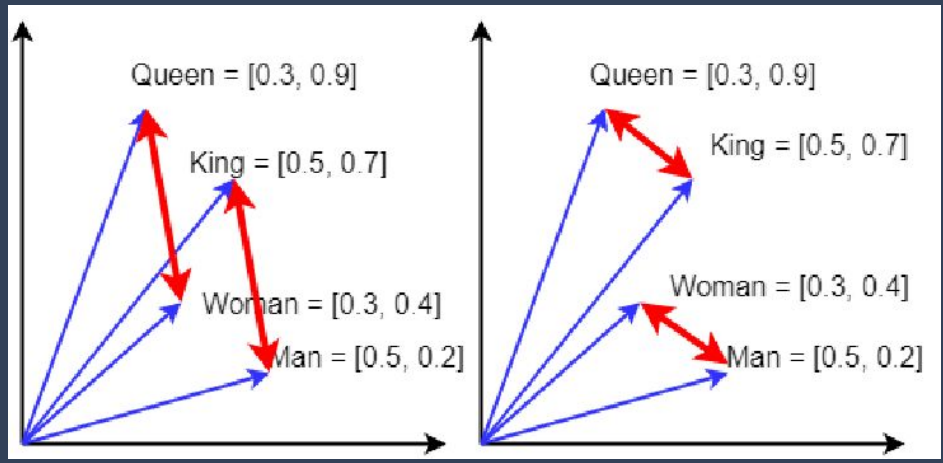
queen - woman + man = king.

Word Embeddings: Why?

(enhancing text representations)

- Word embeddings capture semantic relationships more efficiently.





queen - woman + man = king.

Word Embeddings: Why?

(enhancing text representations)

- Word embeddings capture semantic relationships more efficiently.
- cos_similarity([0.1, 0.3, ...0.2], [0.2, 0.3, ...0.5])

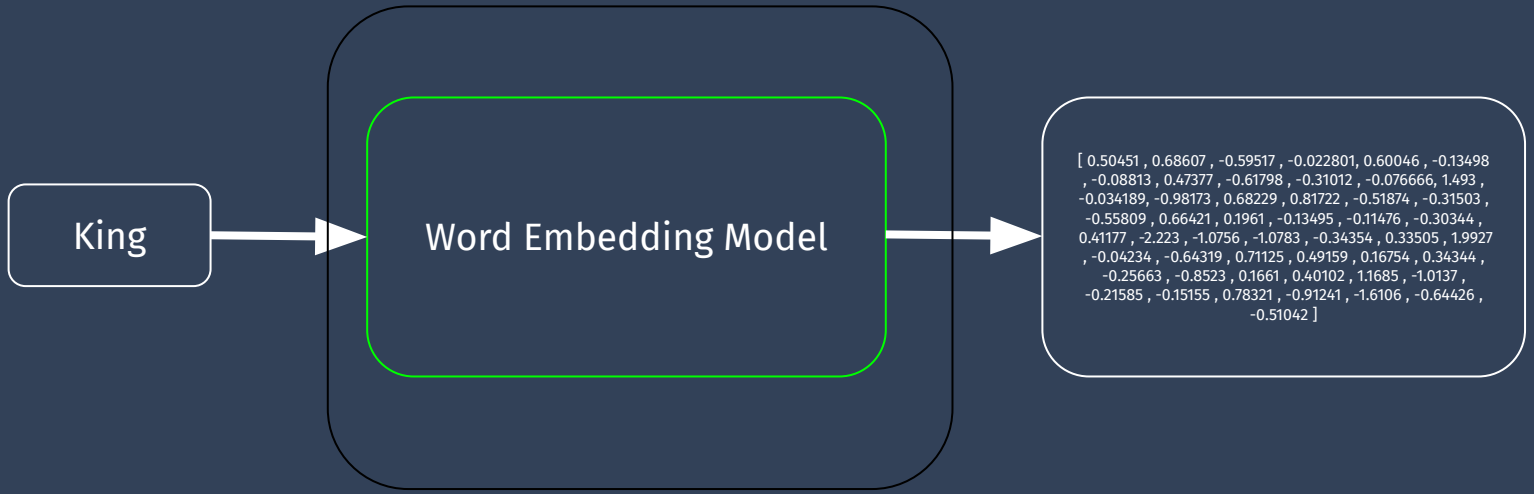
Word Embeddings

What?

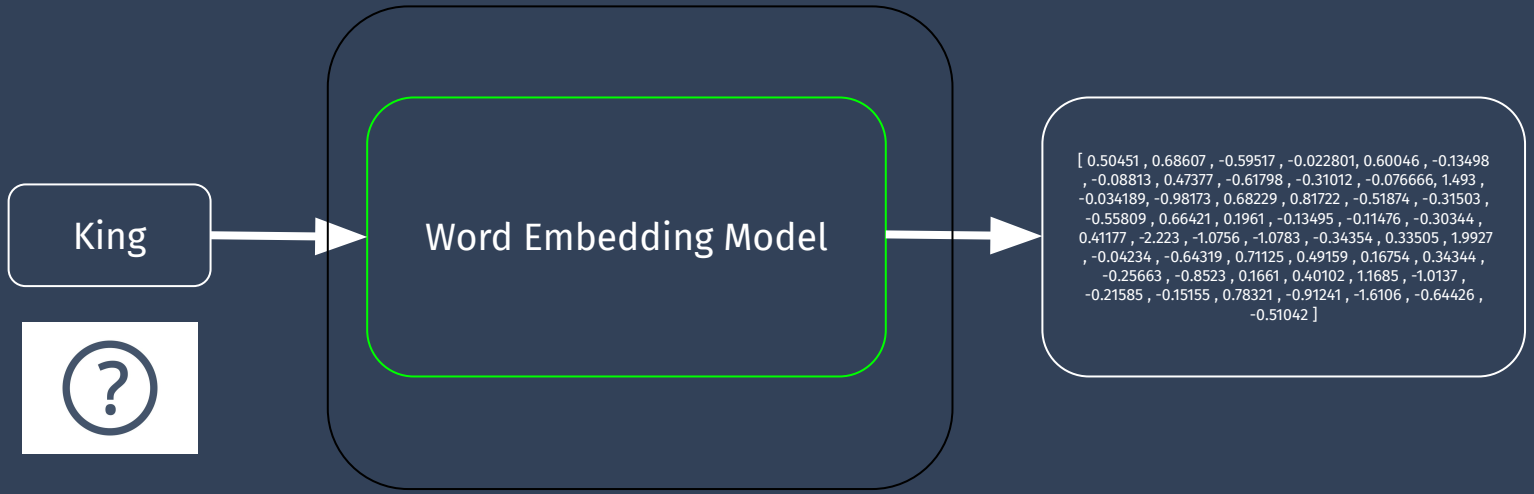
Why?

How?

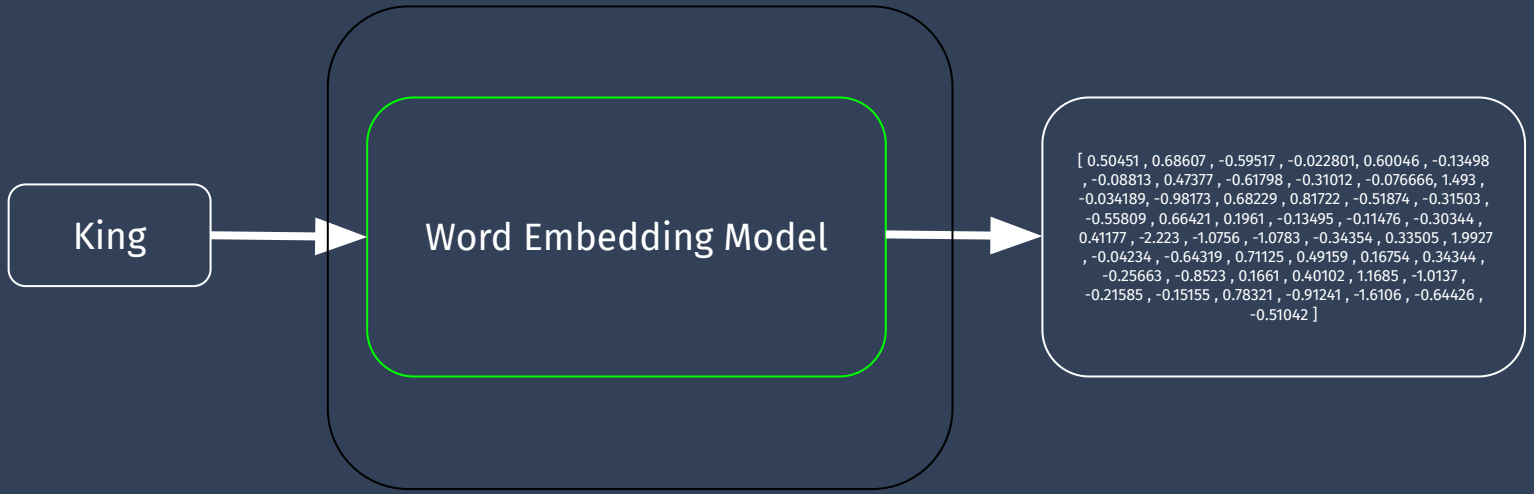
Word Embeddings: How?



Word Embeddings: How? Input



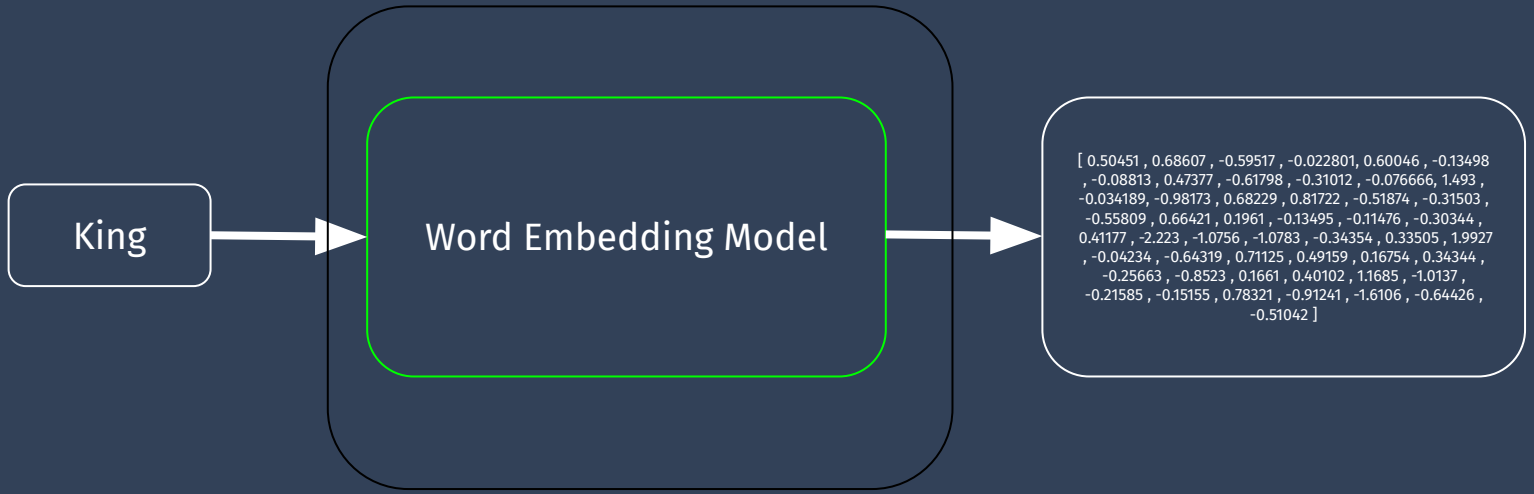
Word Embeddings: How? Input?



One-hot encoding:

[car, seat, ... king,]
[0 ,0 ,0, ..., 1, , 0, ...]

Word Embeddings: How? Input?

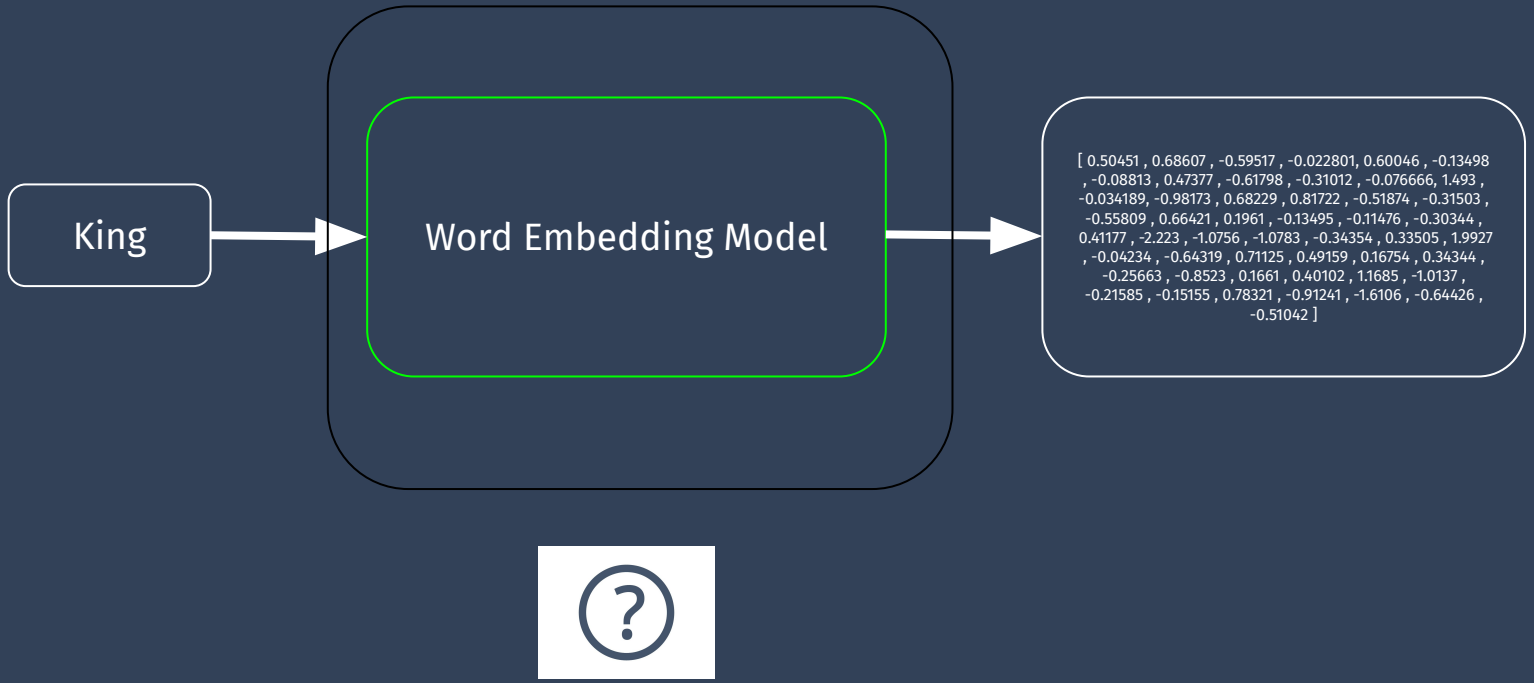


One-hot encoding:

[car, seat, ... king,]
[0 ,0 ,0, ..., 1, , 0, ...]

One-hot Encoding

Word Embeddings: How? Input



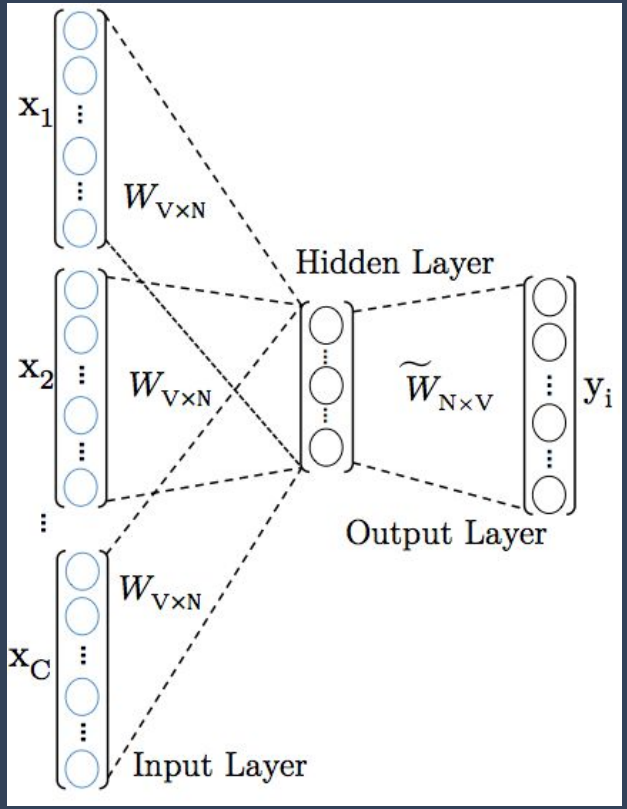
Word Embeddings: How? Language Model

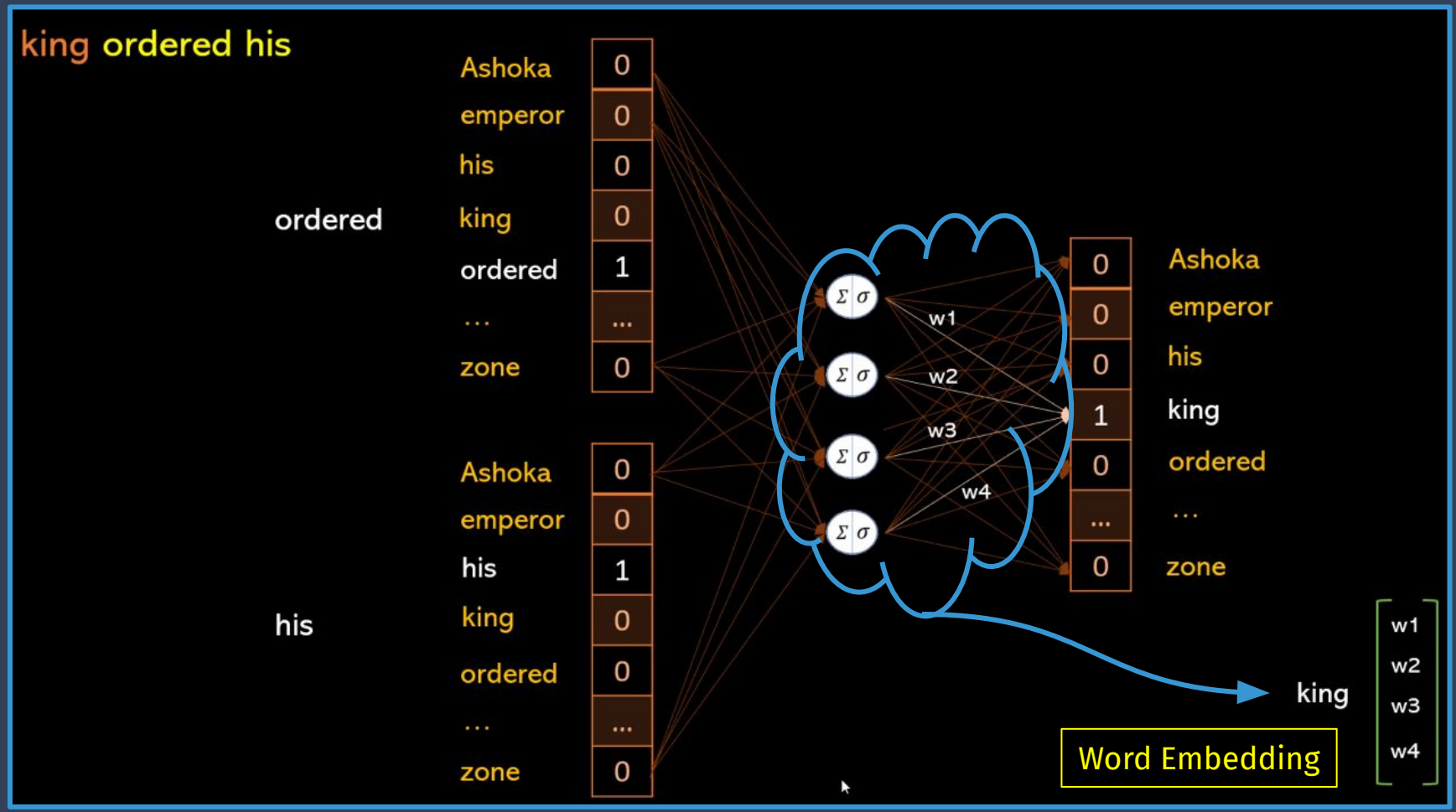
- A supervised model → It needs training data
- Using context!

  “It has been a long time. **How** are you?”

  <“It has been a long time. _____ are you?”, “How”>

Word Embeddings: Language Model





Word Embeddings:How? Language Model

- Using context (auto-regression, semi-supervised learning) is one secret sauce for the recent advancement in NLP.
- Why?

Word Embeddings

- We talked about:
  - Why word embeddings
  - The idea of auto-regression, semi-supervised models
- Now let’s talk about one specific word embedding model
  - word2vec

Word Embeddings: word2vec

- It’s a shallow neural network model that learns word associations from a large corpus of text (books, articles, wikipedia).
- It predicts words (word masking!) considering their surroundings

Let’s understand with an example

- Make a neural network that can predict the probability of the target word.
-

An Example:

_____ are you? It has been a long time.

Which word is more likely?

Vocabulary → [How, What, Why, Who, When, :), :p, Where, ...]

Probability → [0.8, 0, 0, 0, 0, 0, 0, 0.2, 0, 0,.....]

Ideal output →[1.0, 0, 0, 0, 0, 0, 0, 0.0, 0, 0,.....]

word2Vec original paper →

A neural probabilistic language model by Bengio et al. [2013], University of Montreal

Thou shalt not make a machine in the likeness of a human mind

Sliding window across running text

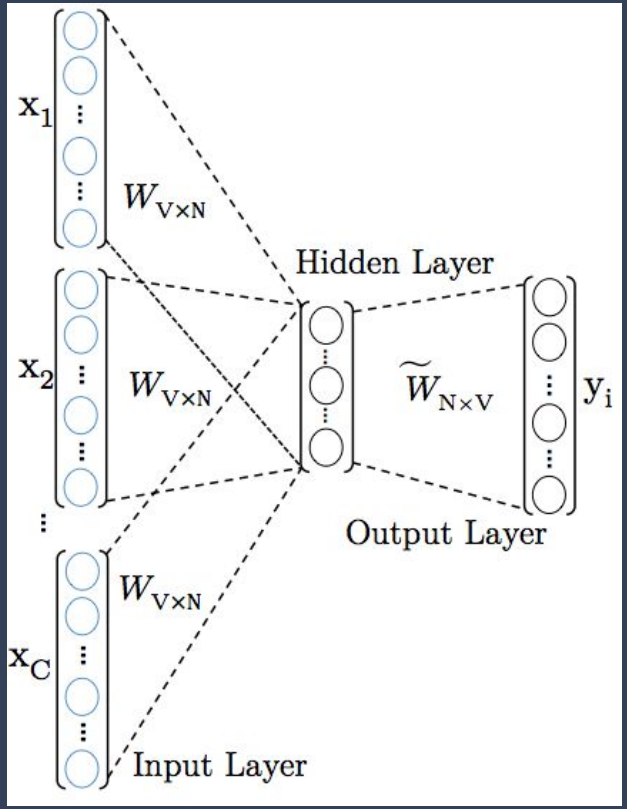
thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	

Dataset

input 1	input 2	output
thou	shalt	not
shalt	not	make
not	make	a
make	a	machine
a	machine	in

Image from https://jalammar.github.io/illustrated-word2vec/

Word2Vec: A step by step Intuition



Word2Vec: A step by step Intuition

Initialization: the weights of the NN are initialized randomly

e.g.
"emperor" -> [0.12, -0.34, -0.56, ...] ← Based on initial weights of the NN model
"lived" -> [-0.78, 0.45, -0.23, ...] ← Based on initial weights of the NN model

The embeddings have no inherent meaning or relationship at the beginning of the training process.

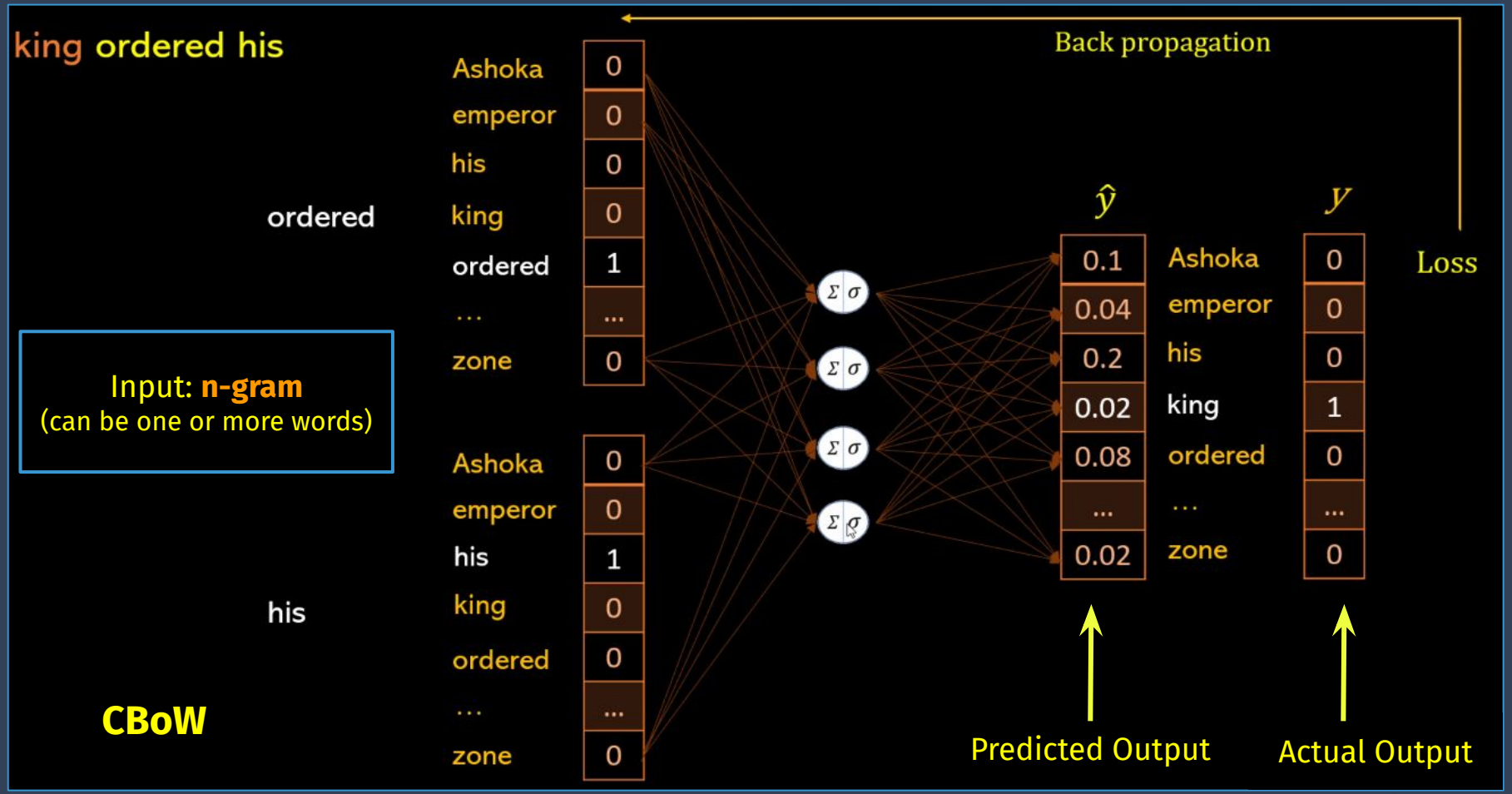
- ***Iterative updates are done***

Word2Vec: A step by step Intuition

- **Semi-Supervised Training**
  - Predict the the target word given the context words (surrounding words).

- Process individual input-output pairs and update the embeddings (weights/parameters of the NN model).

However, at this early stage, the random initial embeddings lack any semantic associations or knowledge about the language, making the predictions less accurate.



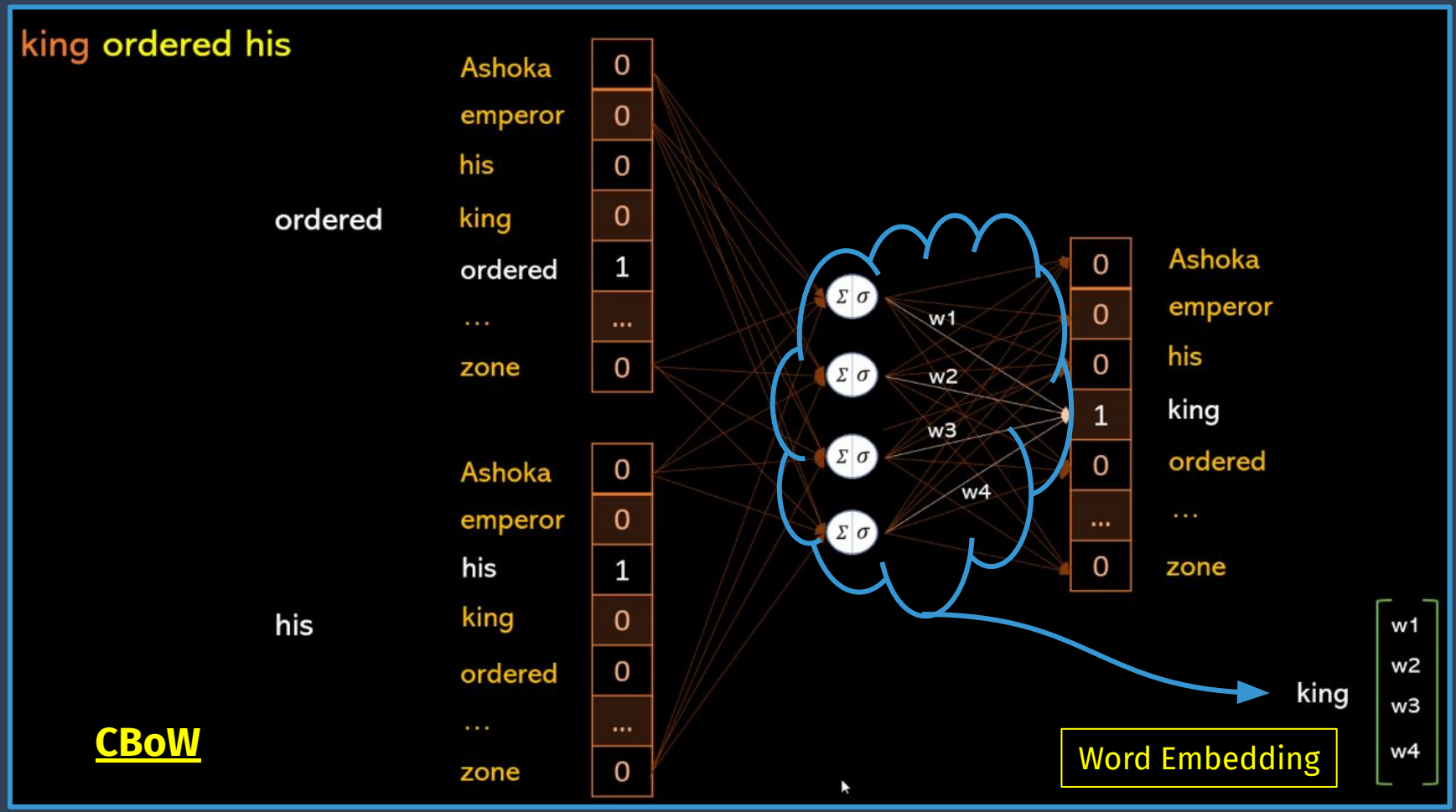
Word2Vec: A step by step Intuition

- **Semi-Supervised Training**
  - Predict the the target word given the context words (surrounding words).

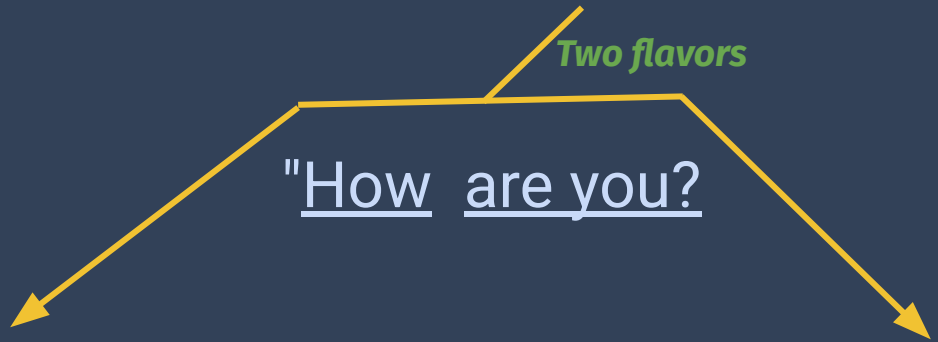
- Process individual input-output pairs and update the embeddings via backpropagation (weights/parameters of the NN model).

Perform repeated iterations with neural networks.

as the training progresses, the model might start to identify that the "target word" often appears in context of the corresponding words with the embedding vector becoming closer to vectors representing similar words.



Word Embeddings: word2vec



CBoW

Input → Context Words: [are, you,?]
Output → Target Word: How

Advantageous when

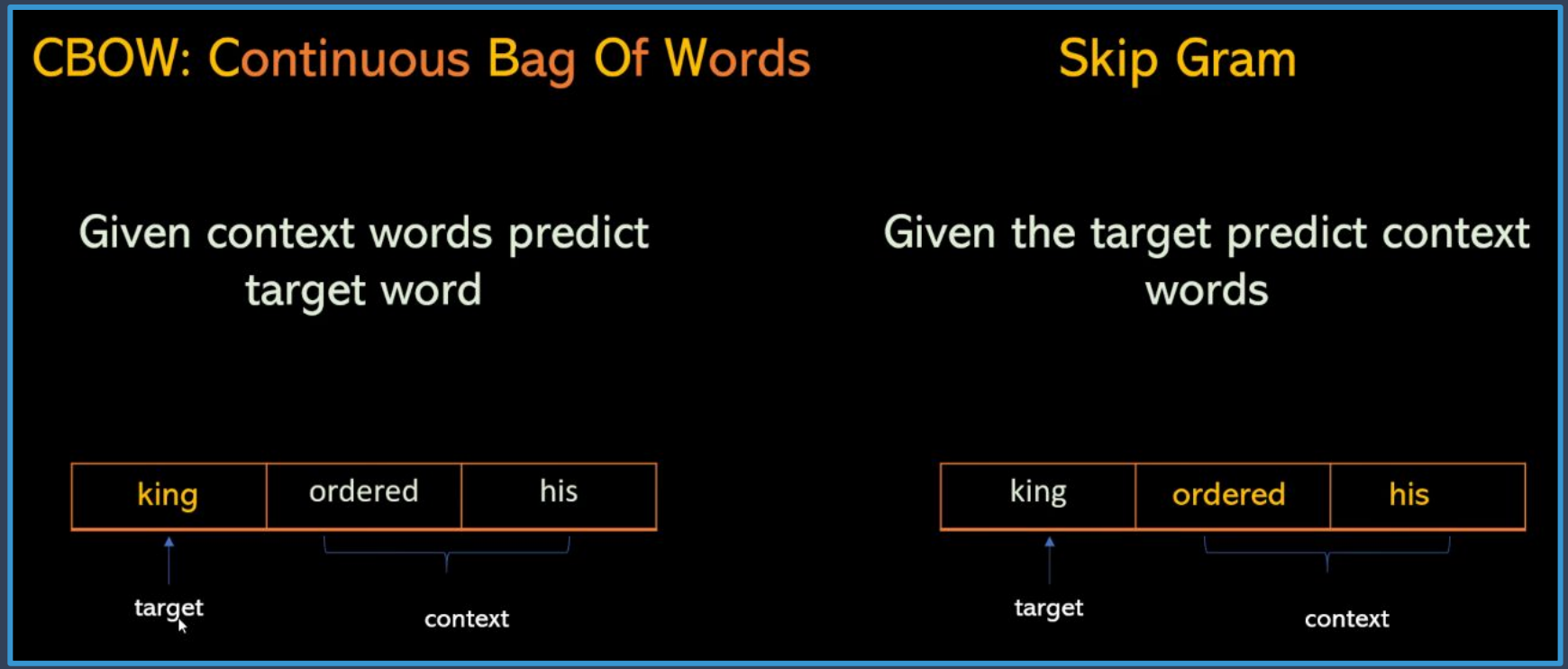
- **Efficiency is a priority**
- **There is a need to obtain representations for more frequent words.**

Skip-gram

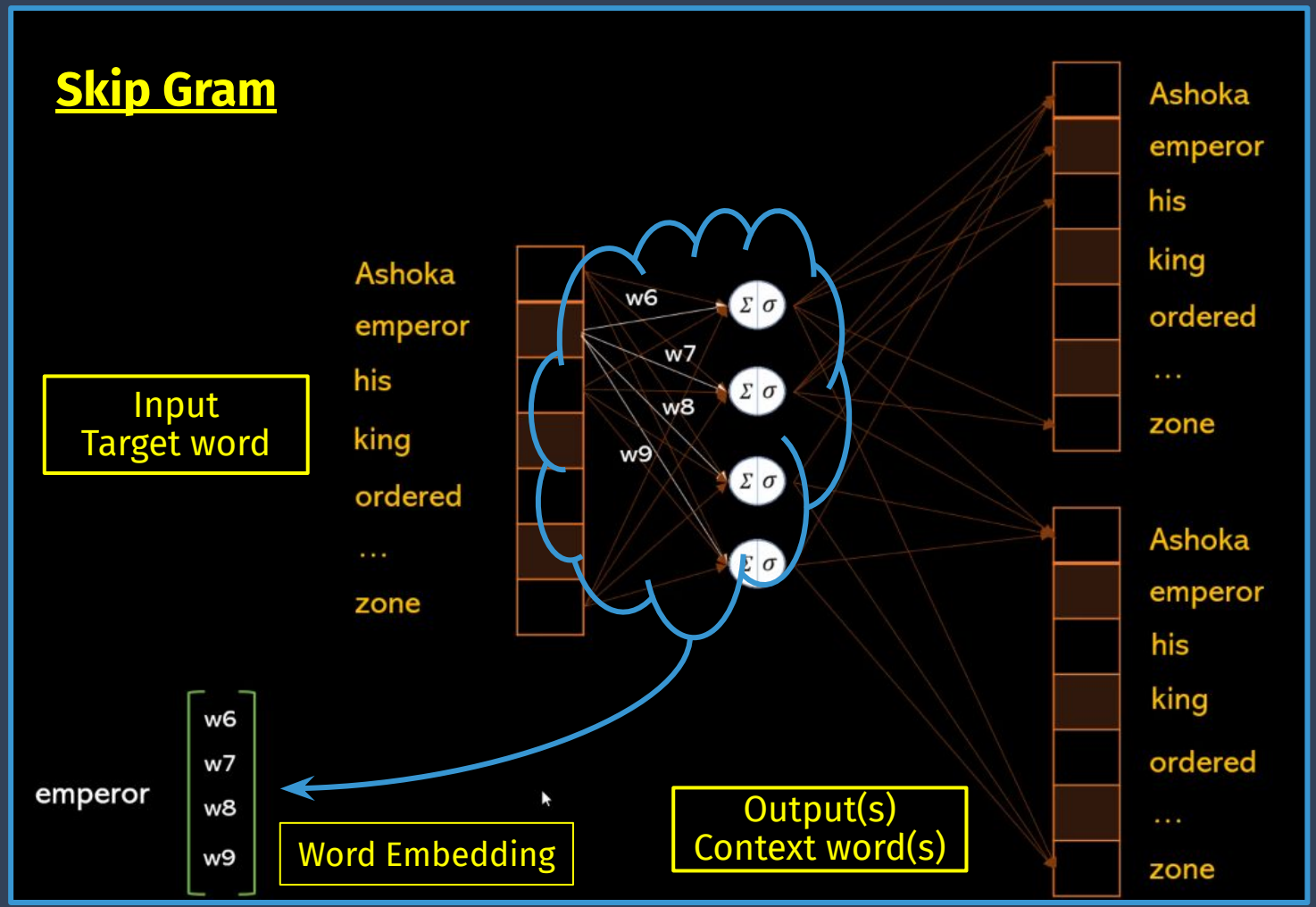
Input → Target Word: How
Output → Context Words: [are, you,?]

more suitable for capturing detailed relationships and context, especially for less frequent words.

CBOW v/s Skip-Gram



Skip Gram


Input
Target word
Ashoka
emperor
his
king
ordered
…
zone
w6
w7
w8
w9
Σ σ
Ashoka
emperor
his
king
ordered
…
zone
Ashoka
emperor
his
king
ordered
…
zone
emperor
w6
w7
w8
w9
Word Embedding
Output(s)
Context word(s)

Food for Thought

1. In what ways can Word2Vec be extended or adapted to handle out-of-vocabulary words or rare words effectively?

1. Can Word2Vec handle polysemy (words with multiple meanings), any strategies that can be used to disambiguate word embeddings in such cases?

1. In what ways can Word2Vec be applied in real-world applications beyond NLP, such as recommendation systems or information retrieval?

1. What role does word order play in the effectiveness of Word2Vec embeddings, and how can you account for it in sequence modeling tasks?

Word2Vec: Two main challenges

- Running gradient descent on a neural network that is very large is going to be slow.

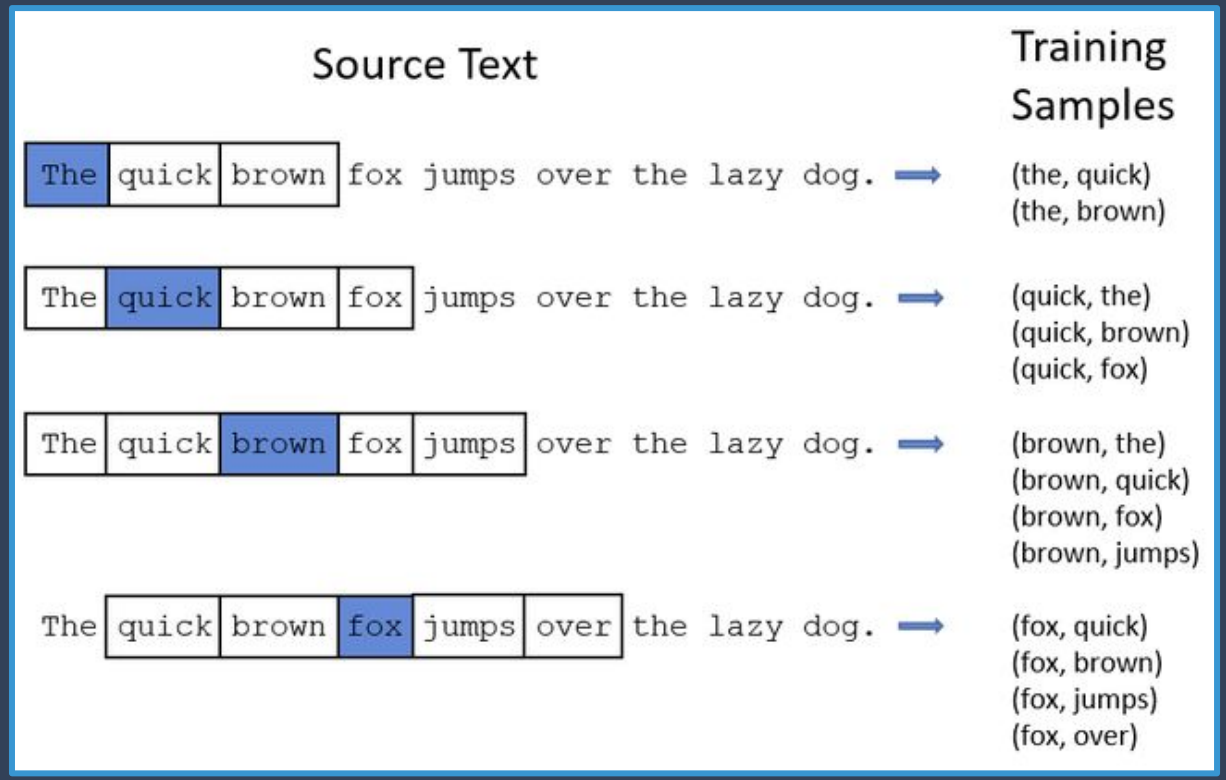
- And to make matters worse, you need a huge amount of training data in order to tune that many weights and avoid over-fitting.
    - millions of weights times billions of training samples means that training this model is going to be a beast.

Word2Vec: Addressing Challenges

- The authors addressed the challenges in their subsequent paper [2013] with the following techniques →
  - Subsampling frequent words to decrease the number of training examples.
  - Negative Sampling
    - Each training sample updates only a small percentage of the model's weights

Word2Vec (making it efficient)

Subsampling frequent words



Word2Vec (making it efficient)

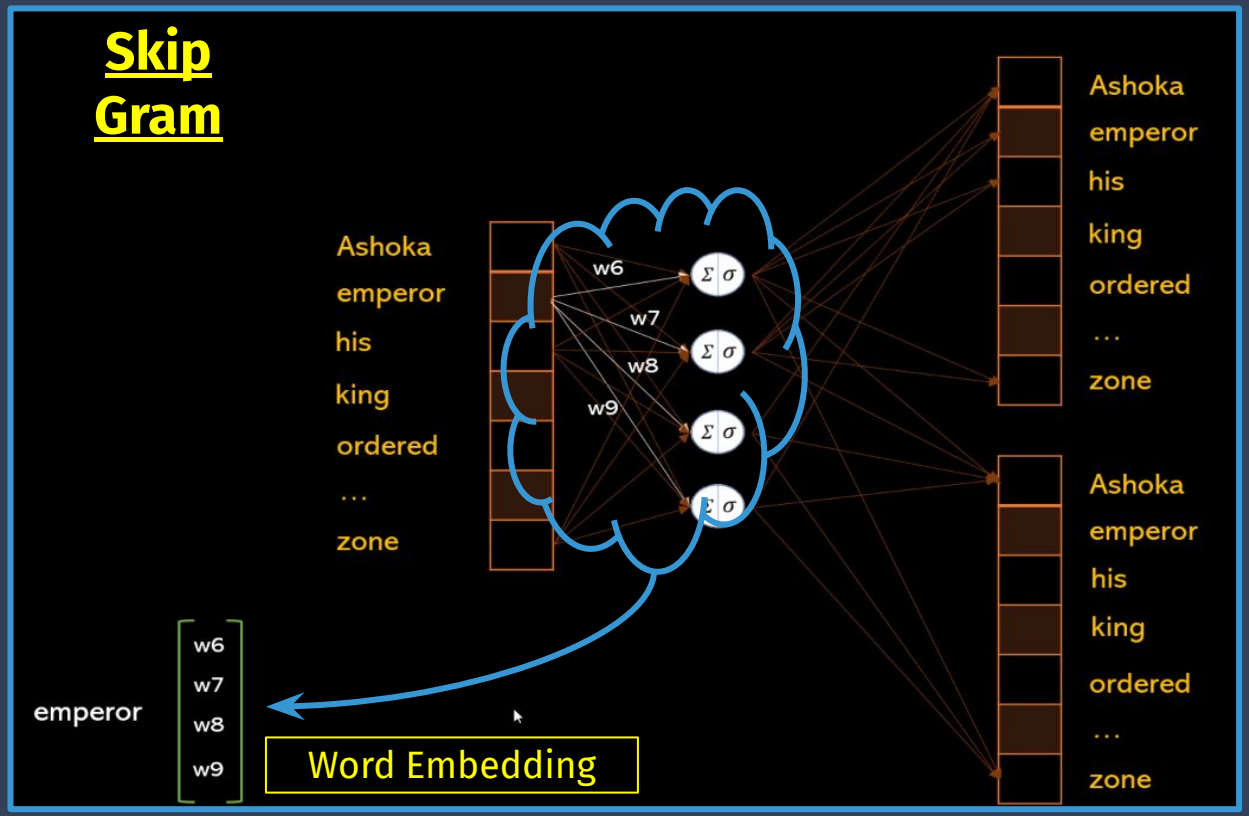
Negative Sampling

- When training the network on the word pair (“fox”, “quick”), recall that the “label” or “correct output” of the network is a one-hot vector. That is, for the output neuron corresponding to “quick” to output a 1, and for all of the other thousands of output neurons to output a 0.

- Negative Sampling→ Randomly select just a small number of “negative” words (let’s say 5) to update the weights for.
  - In this context, a “negative” word is one for which we want the network to output a 0 for).
  - Take note that we also update the weights for our “positive” word (which is the word “quick” in our current example).

Word2Vec (making it efficient)

Negative Sampling



Word2Vec (making it efficient)

Words Pairs and Phrases

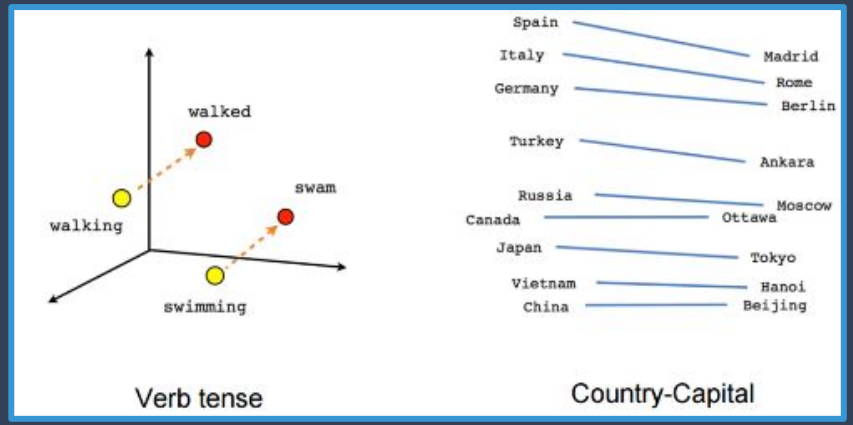
- Use 2-grams (& phrases) to capture better semantic associations
  - Consider “New_York” or “New_York_City”
    - The individual words have a much different meaning

- The addition of phrases to the model swells the vocabulary size but provides better semantic embeddings.

Implications

King – man + woman = Queen

USA – Washington D.C + Delhi = India



Word2Vec: Important Points to remember

- In summary, a look-up table
  - Disadvantage: E.g., The word "bank" will get the same embeddings in the following two cases: "I went to the bank to deposit the cheque" and "I crossed the river bank"

- Choice of model architecture
  - Skip-gram tends to perform better for infrequent words and phrases, while CBOW is faster and often works well for more frequent words.
    - Small corpus → Skip-gram
    - Large corpus → CBOW (faster)

- Dense embeddings
  - Overcomes curse of dimensionality with simplicity

- Self-Supervised: Easy to get data and can be computed relatively quickly.
  - Pre-trained embeddings are used in a lot of applications.

Word2Vec → Limitations and Extensions

- Global information is not preserved in Word2Vec
  - Solved by GloVe
- Doesn’t work well for morphologically rich languages
  - Solved by FastText / MUSE
- Lacks broad context awareness
  - Solved by LSTMs, BERT, GPT

Questions?


Thank
you